



# Noritake itron GU-3000B Evaluation Library

---

|                  |             |
|------------------|-------------|
| Document Number: | E-M-0123-00 |
| Issue Date:      | 07/26/2012  |

|                                                                                                                                                                                                                                                                                  |                                                                                                                                                                           |                                                                                                                                                                                                |                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Noritake Co., Inc.<br/>Electronics Division Headquarter</b><br>2635 Clearbrook Drive<br>Arlington Heights, IL 60005<br>Toll free: (800) 779 - 5846<br>Phone: (847) 439 - 9020<br>support.ele@noritake.com<br><a href="http://www.noritake-elec.com">www.noritake-elec.com</a> | <b>East Coast<br/>New Jersey Branch</b><br>15-22 Fair Lawn Ave.<br>Fair Lawn, NJ 07410<br>Toll free: (888) 296 - 3423<br>Phone: (201) 475 - 5200<br>Fax: (201) 796 - 2269 | <b>Midwest, Canada, and Mexico<br/>Chicago Branch</b><br>2635 Clearbrook Dr.<br>Arlington Heights, IL 60005<br>Toll free: (800) 779 - 5846<br>Phone: (847) 439 - 9020<br>Fax: (847) 593 - 2285 | <b>West Coast<br/>Los Angeles Branch</b><br>21081 S. Western Ave. Ste 180<br>Torrance, CA 90501<br>Toll free: (888) 795 - 3423<br>Phone: (310) 320 - 1700<br>Fax: (310) 320 - 2900 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

You must agree these terms and conditions. This software is provided by Noritake Co., Inc. "AS IS" and any express or implied warranties, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose are disclaimed. In no event shall the copyright owner or contributors be liable for any direct, indirect, incidental, special, exemplary, or consequential damages (including, but not limited to, procurement of substitute goods or services; loss of use, data, or profits; or business interruption) however caused and on any theory of liability, whether in contract, strict liability, or tort (including negligence or otherwise) arising in any way out of the use of this software, even if advised of the possibility of such damage.

If this document is distributed with software that includes an end user agreement, this document, as well as the software described in it, is furnished under license and may be used or copied only in accordance with the terms of such license. Except as permitted by any such license, no part of this document may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, recording, or otherwise, without the prior written permission of Noritake Co., Inc. Please note that the content in this document is protected under copyright law even if it is not distributed with software that includes an end user license agreement.

The content of this document is furnished for informational use only, is subject to change without notice, and should not be construed as a commitment by Noritake Co., Inc. Noritake Co., Inc. assumes no responsibility or liability for any errors or inaccuracies that may appear in the informational content contained in this document.

Any references to company names in sample codes are for demonstration purposes only and are not intended to refer to any actual organization.

Noritake and Itron are either registered trademarks or trademarks of Noritake Co., Inc. in the United States and/or other countries.

© 2012 Noritake Co., Inc. All rights reserved

Noritake Co., Inc., 2635 Clearbrook Drive, Arlington Heights, IL 60005, USA.

## Contents

---

- [Using the Library](#)
  - [About the Library](#)
  - [Supported Functionality](#)
  - [Module Setup](#)
  - [Installation](#)
  - [Configuration](#)
- [Type and Method Reference](#)
- [Appendix A](#)

## Using the Library

---

### About the Library

This library provides access to the base functionality of the GU-3000B series modules using an Atmel® 8-bit AVR microcontroller or a Linux system.

For Atmel® AVR, code that implements the RS-232 interface to communicate with the module is included. For Linux, code to set up a serial device driver is included. The parallel interface is not included in this evaluation library.

This library is targeted at the modules with the module number in the form:

GU\*\*\*X\*\*\*\*-39\*\*B

Example

- GU128X128D-3900B
- GU256X128C-3900B
- GU256X128D-3900B
- GU256X16M-3900B
- GU256X64C-3900B
- GU256X64D-3900B
- GU256X64E-3900B
- GU384X32L-3900B
- GU512X32H-3900B

## Supported Functionality

Because the GU-3000 has many functions that serve a variety of use cases and code space on microcontrollers are limited, this library only supports a subset the commands.

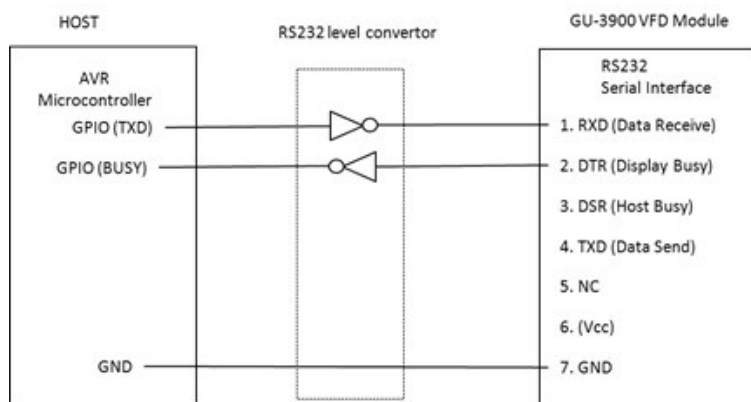
See [Appendix A](#) for a list of the supported commands.

Roughly, this library supports the direct command modes.

User setup mode, direct memory access mode, and memory rewrite mode are not supported by this release.

## Module Setup

For Atmel® AVR, connect the module according to the following diagram:

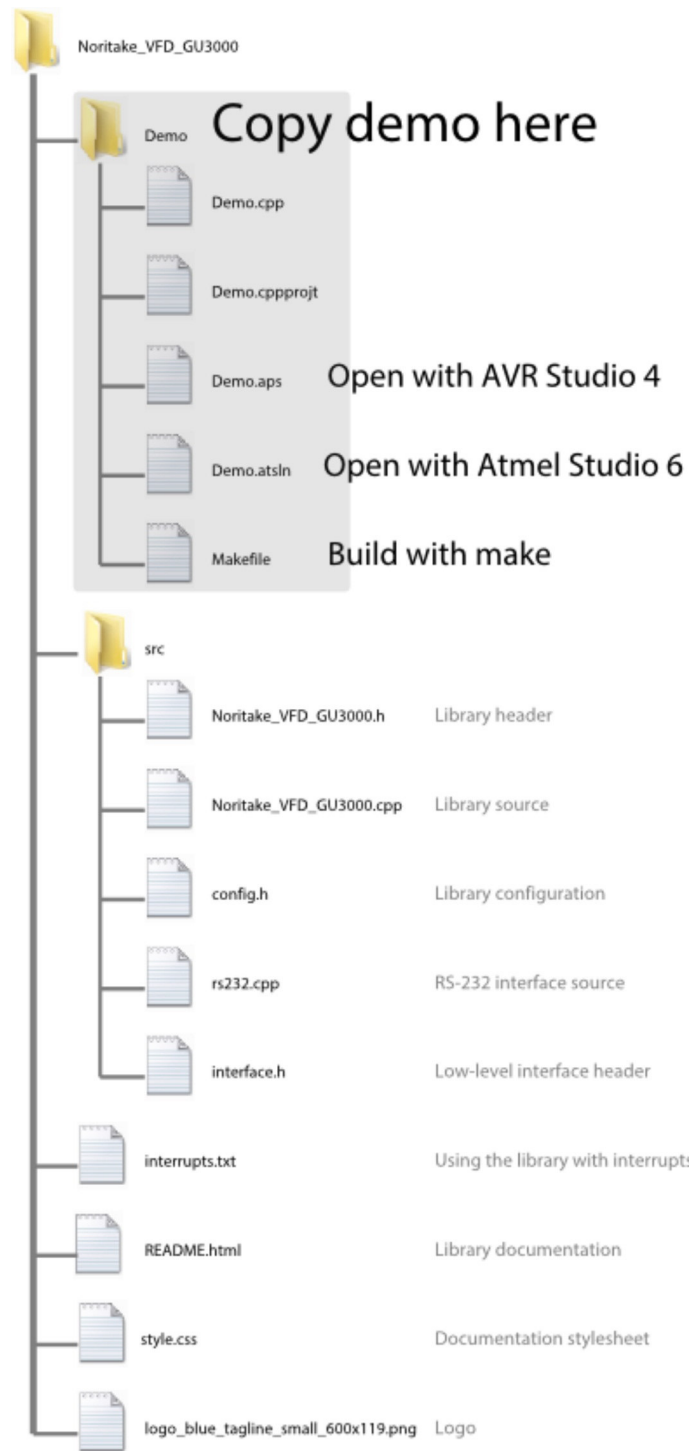


Note that the TXD signal from the host and the BUSY signal from the module must be RS-232 levels and inverted.

## Installation

1. Download the [code library](#) .
2. Unzip the library file to your work area.

3. Extract the demo folder into the folder for the GU-3000 code library.
4. Set the configuration options in `config.h` included with the code library. See the *Configuration* section.
5. Open the *Demo.aps* project file with AVR Studio 4 or *Demo.atsln* in Atmel Studio 6 or build *Makefile* on Linux.



## Configuration

The GU-3000 code library can work in a variety of hardware settings. The library is configured by setting preprocessor values in the *config.h* file in the *src* directory of the library. The configuration options in the code library's *config.h* must match the host system's hardware setup.

The library is designed so that all of its source files are compiled even if they are not used. For example, when set up to use the Linux serial device interface, the Atmel® AVR serial interface files will automatically disable themselves based on the options set in *config.h*.

## Mandatory Options

There are a few mandatory options that must be set regardless of the hardware setup.

| Name                            | Description                                                                                                                                                                                                                                                                                    | Example                     |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| <b>F_CPU</b>                    | CPU frequency in Hertz.<br>This is only used for Atmel® AVR.                                                                                                                                                                                                                                   | 16000000UL<br>for 16MHz     |
| <b>NORITAKE_VFD_RESET_DELAY</b> | The delay time in milliseconds before beginning communication with the module. This allows the VFD module to start up as well as avoids program restarts due to in-circuit debuggers.<br><br>This value will vary depend on the power supply and hardware setup. 500ms is generally sufficient | 500 for 500ms               |
| <b>NORITAKE_VFD_HEIGHT</b>      | Height of the display in dots. This number is given after the “X” in the model number:<br>GU***X__*~3***                                                                                                                                                                                       | 128 for<br>GU256X128C-3900B |
| <b>NORITAKE_VFD_WIDTH</b>       | Width of the display in dots. This number is given before the “X” in the model number:<br>GU__X***~3***                                                                                                                                                                                        | 256 for<br>GU256X128C-3900B |

## RS-232 Serial Interface

| Name                            | Description                                       | Example                                                   |
|---------------------------------|---------------------------------------------------|-----------------------------------------------------------|
| <b>NORITAKE_VFD_INTERFACE</b>   | 0 for serial interface                            |                                                           |
| <b>OUT_PORT &amp; OUT_PIN</b>   | Output signal from host to module (RXD on module) | PORTA & 0 when output is connected to pin 0 on port A     |
| <b>BUSY_PORT &amp; BUSY_PIN</b> | Module busy signal (DTR on module)                | PORTA & 1 when output pin is connected to pin 1 on port A |

## Linux Serial Device Interface

The Linux serial device interface uses Linux character device files to interface with the module.

**Make sure logged in user has read and write access to the device file before using the library.**

You can give non-root users access to the device file with:

```
chmod o+rw /dev/ttyUSB0
```

You must be root or running with elevated privilege through `sudo` or `su` to run this command.

| Name                   | Description                                          | Example            |
|------------------------|------------------------------------------------------|--------------------|
| NORITAKE_VFD_INTERFACE | 1 for Linux serial device serial interface           |                    |
| NORITAKE_VFD_FILE      | The path to the device file connected to the module. | /dev/ttyUSB0       |
| NORITAKE_VFD_BAUD      | The baud rate of the device.                         | 38400 for 38400bps |

## Type and Method Reference

---

### Index

- Types
  - [FontStyle](#)
  - [Shape](#)
  - [ImageMemoryArea](#)
  - [WipeDirection](#)
  - [ScrollMode](#)
  - [CompositionMode](#)
  - [FontFormat](#)
  - [CustomCharacter](#)
  - [AsciiVariant](#)
  - [Charset](#)
  - [MultibyteCharset](#)
- Printing Methods
  - `void print(char c);`
  - `void print(const char *str);`
  - `void print(const char *buffer, size_t size);`
  - `void print(int number, uint8_t base);`
  - `void print(unsigned number, uint8_t base);`
  - `void print(long number, uint8_t base);`
  - `void print(unsigned long number, uint8_t base);`
  - `void print(unsigned x, uint8_t y, char c);`
  - `void print(unsigned x, uint8_t y, const char *str);`
  - `void print(unsigned x, uint8_t y, const char *buffer, uint8_t len);`
  - `void print(unsigned x, uint8_t y, int number, uint8_t base);`
  - `void print(unsigned x, uint8_t y, unsigned number, uint8_t base);`
  - `void print_p(const char *str);`
  - `void print_p(unsigned x, uint8_t y, const char *str);`
  - `void print_p(unsigned x, uint8_t y, const char *buffer, uint8_t len);`
  - `void println(char c);`
  - `void println(const char *str);`
  - `void println(const char *buffer, size_t size);`
  - `void println(int number, uint8_t base);`
  - `void println(unsigned number, uint8_t base);`
  - `void println(long number, uint8_t base);`
  - `void println(unsigned long number, uint8_t base);`
- Character-Based
  - `void GU3000_back();`
  - `void GU3000_forward();`
  - `void GU3000_lineFeed();`
  - `void GU3000_home();`
  - `void GU3000_carriageReturn();`
  - `void GU3000_setCursor(unsigned x, unsigned y);`
  - `void GU3000_clearLine();`

- void GU3000\_clearLineEnd();
- void GU3000\_cursorOn();
- void GU3000\_cursorOff();
- **Module Control**
  - void GU3000\_clearScreen();
  - void GU3000\_init();
  - void GU3000\_setScreenBrightness(unsigned level);
  - void GU3000\_wait(uint8\_t time);
  - void GU3000\_shortWait(uint8\_t time);
  - void GU3000\_displayOn();
  - void GU3000\_displayOff();
  - void GU3000\_displayOff(uint8\_t timer);
- **Fonts**
  - void GU3000\_setFontStyle(FontStyle style);
  - void GU3000\_setFontSize(FontFormat format, uint8\_t x, uint8\_t y);
  - void GU3000\_boldOn();
  - void GU3000\_boldOff();
  - void GU3000\_useCustomChars(bool enable);
  - void GU3000\_defineCustomChar(unsigned code, FontFormat format, const uint8\_t \*data);
  - void GU3000\_deleteCustomChar(unsigned code, FontFormat format);
- **Encoding**
  - void GU3000\_useMultibyteChars(bool enable);
  - void GU3000\_setMultibyteCharset(uint8\_t code);
  - void GU3000\_setAsciiVariant(AsciiVariant code);
  - void GU3000\_setCharset(Charset code);
- **Screen Effects**
  - void GU3000\_invertOn();
  - void GU3000\_invertOff();
  - void GU3000\_setCompositionMode(CompositionMode mode);
  - void GU3000\_setScrollMode(ScrollMode mode);
  - void GU3000\_setHorizScrollSpeed(uint8\_t speed);
  - void GU3000\_scrollScreen(unsigned x, unsigned y, unsigned count, uint8\_t speed);
  - void GU3000\_blinkScreen(bool enable, bool reverse, uint8\_t on, uint8\_t off, uint8\_t times);
  - void GU3000\_wipeScreen(WipeDirection direction, unsigned speed, uint8\_t filler);
  - void GU3000\_reverseWipeScreen(WipeDirection direction, unsigned speed, unsigned addr);
  - void GU3000\_disolveScreen(unsigned speed, unsigned addr);
- **Drawing**
  - void GU3000\_drawImage(unsigned width, uint8\_t height, const uint8\_t \*data);
  - void GU3000\_drawImage\_p(unsigned width, uint8\_t height, const uint8\_t \*data);
  - void GU3000\_drawImage(ImageMemoryArea area, unsigned long address, uint8\_t srcHeight, unsigned width, uint8\_t height);
  - void GU3000\_drawImage(unsigned x, uint8\_t y, ImageMemoryArea area, unsigned long address, uint8\_t srcHeight, unsigned width, uint8\_t height, unsigned offsetx, unsigned offsety);
  - void GU3000\_drawImage(unsigned x, uint8\_t y, ImageMemoryArea area, unsigned long address, unsigned width, uint8\_t height);
  - void GU3000\_drawImage\_p(unsigned x, uint8\_t y, unsigned width, uint8\_t height, const uint8\_t \*data);
  - void GU3000\_drawImage(unsigned x, uint8\_t y, unsigned width, uint8\_t height, const uint8\_t \*data);
  - void GU3000\_scrollImage(ImageMemoryArea area, unsigned long address, uint8\_t srcHeight, unsigned width, uint8\_t height, uint8\_t speed);
  - void GU3000\_dot(unsigned x, unsigned y, bool on=true);
  - void GU3000\_shape(Shape shape, unsigned x0, unsigned y0, unsigned x1, unsigned y1, bool on=true);
  - void GU3000\_defineImage(ImageMemoryArea area, unsigned addr, unsigned width, uint8\_t height, const uint8\_t \*data);
  - void GU3000\_defineImage\_p(ImageMemoryArea area, unsigned addr, unsigned width, uint8\_t height, const uint8\_t \*data);
- **Window and Screen**
  - void GU3000\_selectWindow(uint8\_t window);
  - void GU3000\_defineWindow(uint8\_t window, unsigned x, unsigned y, unsigned width, unsigned height);

- [void GU3000\\_deleteWindow\(uint8\\_t window\);](#)
- [void GU3000\\_joinScreens\(\);](#)
- [void GU3000\\_separateScreens\(\);](#)

## Types

### FontStyle

Identifies the font style: fixed or proportional.

#### Users

- [GU3000\\_setFontStyle\(FontStyle style\)](#)

#### Parameters

|                   |                                                                                              |
|-------------------|----------------------------------------------------------------------------------------------|
| FixedFont         | 0: each character takes the same space                                                       |
| ProportionalFont1 | 1: each character has a specific width; 1 space is inserted on the right                     |
| ProportionalFont2 | 2: each character has a specific width; 1 space is inserted on the right and left            |
| ProportionalFont3 | 3: each character has a specific width; 2 spaces are inserted on the right and 1 to the left |

### Shape

Identifies a shape.

#### Users

- [GU3000\\_shape\(Shape shape, unsigned x0, unsigned y0, unsigned x1, unsigned y1, bool on=true\)](#)

#### Parameters

|                 |                                                              |
|-----------------|--------------------------------------------------------------|
| LineShape       | 0: draw a line from (x0, y0) to (x0, y0)                     |
| HollowRectShape | 1: draw the outline of a rectangle from (x0, y0) to (x0, y0) |
| FilledRectShape | 2: fill in a rectangle from (x0, y0) to (x0, y0)             |

### ImageMemoryArea

Identifies the memory area to use in an operation.

#### Users

- [GU3000\\_drawImage\(ImageMemoryArea area, unsigned long address, uint8\\_t srcHeight, unsigned width, uint8\\_t height\)](#)
- [GU3000\\_drawImage\(unsigned x, uint8\\_t y, ImageMemoryArea area, unsigned long address, uint8\\_t srcHeight, unsigned width, uint8\\_t height, unsigned offsetx, unsigned offsety\)](#)
- [GU3000\\_drawImage\(unsigned x, uint8\\_t y, ImageMemoryArea area, unsigned long address, unsigned width, uint8\\_t height\)](#)

- [GU3000\\_defineImage\(ImageMemoryArea area, unsigned addr, unsigned width, uint8\\_t height, const uint8\\_t \\*data\)](#)
- [GU3000\\_defineImage\\_p\(ImageMemoryArea area, unsigned addr, unsigned width, uint8\\_t height, const uint8\\_t \\*data\)](#)
- [GU3000\\_scrollImage\(ImageMemoryArea area, unsigned long address, uint8\\_t srcHeight, unsigned width, uint8\\_t height, uint8\\_t speed\)](#)

#### Parameters

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| RAMImageArea    | 0: RAM on the module; contents cleared when the module is reset |
| FlashImageArea  | 1: FlashROM on the module; contents until it is overwritten     |
| ScreenImageArea | 2: reflects the contents of the screen                          |

## WipeDirection

Identifies the motion of a wipe.

#### Users

- [GU3000\\_wipeScreen\(WipeDirection direction, unsigned speed, uint8\\_t filler\)](#)
- [GU3000\\_reverseWipeScreen\(WipeDirection direction, unsigned speed, unsigned addr\)](#)

#### Parameters

|              |                                                    |
|--------------|----------------------------------------------------|
| RightToLeft  | 0: Wipe begins from the right end and steps left   |
| LeftToRight  | 1: Wipe begins from the left end and steps right   |
| OpenCurtain  | 2: Wipe begins from the center and expands outward |
| CloseCurtain | 3: Wipe begins from the edges and expands inward   |

## ScrollMode

Identifies scrolling and wrapping behavior.

#### Users

- [GU3000\\_setScrollMode\(ScrollMode mode\)](#)

#### Parameters

|                 |                                                                                                                                                                                                                           |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WrappingMode    | 1: When the cursor reaches the right end of the screen, the cursor moves to the left end of the next line. If on the last line, the cursor moves to the home position (0, 0).                                             |
| VertScrollMode  | 2: When the cursor reaches the right end of the screen, the cursor moves to the left end of the next line. If on the last line, the screen is moved up by 8 dots. The bottom 8 dots are cleared. The top 8 dots are lost. |
| HorizScrollMode | 3: When the cursor reaches the right end of the screen, the current line is shifted left by the size of one character. The space to the right is cleared. The cursor is not moved.                                        |

[GU3000\\_lineFeed\(\)](#) is ignored in this mode.

|                      |                                                                                                                                                                                                                                                                                              |                                            |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| RightAlignScrollMode | 5: Before printing the cursor is moved within one character of the right end of the screen. The current line is shifted left by the size of one character. The space to the right is cleared. The cursor is set to fit exactly one of the printed character on the right edge of the screen. | GU3000_lineFeed() is ignored in this mode. |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|

## CompositionMode

Identifies the way that new images and text are combined with the contents that are already on the screen.

### Users

- [GU3000\\_setCompositionMode\(CompositionMode mode\)](#)

### Parameters

|                       |                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------|
| NormalCompositionMode | 0: screen image is cleared and the source appears exactly                             |
| OrCompositionMode     | 1: dots are lit if either the source or screen image specify it lit                   |
| AndCompositionMode    | 2: dots are lit only if both the source and screen image specify it lit               |
| XorCompositionMode    | 2: dots are lit only if either the source or screen image specify it lit but not both |

## FontFormat

Identifies the size and data format

### Users

- [GU3000\\_setFontSize\(FontFormat format, uint8\\_t x, uint8\\_t y\)](#)
- [GU3000\\_defineCustomChar\(unsigned code, FontFormat format, const uint8\\_t \\*data\)](#)
- [GU3000\\_deleteCustomChar\(unsigned code, FontFormat format\)](#)

### Parameters

|              |                                                                                                                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------|
| _6x8Format   | 1: 6×8 font                                                                                                                           |
| CUUFormat    | 0x81: 5×8 font. This format is only for defining 6×8 custom characters in a different format; it does not exist on the module, itself |
| _8x16Format  | 2: 8×16 font                                                                                                                          |
| _12x24Format | 3: 12×24 font (ASCII and Japanese only)                                                                                               |
| _16x32Format | 4: 16×32 font                                                                                                                         |
| _16x16Format | 0x10: 16×16 font for defining multibyte characters                                                                                    |
| _32x32Format | 0x14: 32×32 font for defining multibyte characters                                                                                    |

## CustomCharacter

Identifies the custom character number for multibyte character set language.

## Users

- [GU3000\\_defineCustomChar\(unsigned code, FontFormat format, const uint8\\_t \\*data\);](#)
- [void GU3000\\_deleteCustomChar\(unsigned code, FontFormat format\);](#)

## Parameters

|                   |        |
|-------------------|--------|
| CustomJapanese    | 0xec40 |
| CustomKorean      | 0xfea1 |
| CustomSimplified  | 0xfea1 |
| CustomTraditional | 0xfea1 |

## AsciiVariant

Identifies national ASCII variants.

## Users

- [GU3000\\_setAsciiVariant\(AsciiVariant code\)](#)

## Parameters

|                   |    |
|-------------------|----|
| AmericaAscii      | 0  |
| FranceAscii       | 1  |
| GermanyAscii      | 2  |
| EnglandAscii      | 3  |
| Denmark1Ascii     | 4  |
| SweeddenAscii     | 5  |
| ItalyAscii        | 6  |
| Spain1Ascii       | 7  |
| JapanAscii        | 8  |
| NorwayAscii       | 9  |
| Denmark2Ascii     | 10 |
| Spain2Ascii       | 11 |
| LatinAmericaAscii | 12 |
| KoreaAscii        | 13 |

## Charset

Identifies code page.

## Users

- [GU3000\\_setCharset\(Charset code\)](#)

### Parameters

|                       |             |
|-----------------------|-------------|
| CP437                 | 0           |
| EuroStdCharset        | 0: CP437    |
| Katakana              | 1           |
| CP850                 | 2           |
| MultilingualCharset   | 2: CP850    |
| CP860                 | 3           |
| PortugeseCharset      | 3: CP860    |
| CP863                 | 4           |
| CanadianFrenchCharset | 4: CP863    |
| CP865                 | 5           |
| NordicCharset         | 5: CP865    |
| CP1252                | 0x10        |
| CP866                 | 0x11        |
| Cyrillic2Charset      | 0x11: CP866 |
| CP852                 | 0x12        |
| Latin2Charset         | 0x12: CP852 |
| CP858                 | 0x13        |

### MultibyteCharset

Identifies a multibyte character set.

#### Users

- [GU3000\\_setMultibyteCharset\(uint8\\_t code\)](#)

### Parameters

|                        |                          |
|------------------------|--------------------------|
| ShiftJIS               | 0: JIS (X0208 Shift-JIS) |
| JapaneseMBCS           | 0: ShiftJIS              |
| KSC5601                | 1: KSC5601-87            |
| KoreanMBCS             | 1: KSC5601               |
| GB2312                 | 2: GB2312-80             |
| SimplifiedChineseMBCS  | 2: GB2312                |
| Big5                   | 3: Big5                  |
| TraditionalChineseMBCS | 3: Big5                  |

### Printing Methods

```
void print(char c);
```

Print a character.

See  
[GU3000\\_setSci](#)  
([ScrollMode](#) m  
information or  
scrolling and  
wrapping.

#### Parameters

|   |                    |
|---|--------------------|
| c | character to print |
|---|--------------------|

**void print(const char \*str);**

Print a string.

Each character is printed with [print\(char c\)](#).

#### Parameters

|     |                 |
|-----|-----------------|
| str | string to print |
|-----|-----------------|

**void print(const char \*buffer, size\_t size);**

Print a buffer of the given size.

Each character is printed with [print\(char c\)](#).

#### Parameters

|        |                               |
|--------|-------------------------------|
| buffer | characters to print           |
| size   | number of characters to print |

**void print(int number, uint8\_t base);**

Print a number in the given base.

Each character is printed with [print\(char c\)](#).

No leading space or zeros are used.

#### Parameters

|        |                                                |
|--------|------------------------------------------------|
| number | number to print                                |
| base   | base to print in: $2 \leq \text{base} \leq 36$ |

Arithmetic with a long, unsigned, or constant larger than INT\_MAX will cause the long, unsigned, or unsigned long overload of this function to be used.

**void print(unsigned number, uint8\_t base);**

Print a number in the given base.

No leading space or zeros are used.

Each character is printed with [print\(char c\)](#).

#### Parameters

|        |                                                |
|--------|------------------------------------------------|
| number | number to print                                |
| base   | base to print in: $2 \leq \text{base} \leq 36$ |

Arithmetic with a long or constant larger than UINT\_MAX will cause the long or unsigned long overload of this function to be used.

### **void print(long number, uint8\_t base);**

Print a number in the given base.

No leading space or zeros are used.

Each character is printed with [print\(char c\)](#).

#### **Parameters**

|        |                                                |
|--------|------------------------------------------------|
| number | number to print                                |
| base   | base to print in: $2 \leq \text{base} \leq 36$ |

Arithmetic with an unsigned long or constant larger than LONG\_MAX will cause the unsigned long overload of this function to be used.

### **void print(unsigned long number, uint8\_t base);**

Print a number in the given base.

No leading space or zeros are used.

Each character is printed with [print\(char c\)](#).

#### **Parameters**

|        |                                                |
|--------|------------------------------------------------|
| number | number to print                                |
| base   | base to print in: $2 \leq \text{base} \leq 36$ |

### **void print(unsigned x, uint8\_t y, char c);**

Print a character from (x, y).

The cursor is not moved.

Characters can be inverted for the duration of the command by sending character '\x11' and disabled by '\x10'. Inverting from [GU3000\\_invertOn\(\)](#) affects the characters printed with this command and can be disabled with '\x10'.

Normal commands cannot be issued inside this command.

#### **Parameters**

|   |                                                                                             |
|---|---------------------------------------------------------------------------------------------|
| x | x coordinate of top-left corner; -1 continues from the last unaligned print                 |
| y | y coordinate of top-left corner                                                             |
| c | character to print<br>0x10 disables inverted characters<br>0x11 enables inverted characters |

### **void print(unsigned x, uint8\_t y, const char \*str);**

Print a string from (x, y).

#### **Parameters**

No more than 255 characters can be printed at once.

|     |                                                                             |
|-----|-----------------------------------------------------------------------------|
| x   | x coordinate of top-left corner; -1 continues from the last unaligned print |
| y   | y coordinate of top-left corner                                             |
| str | string to print                                                             |

See [print\(unsigned x, uint8\\_t y, char c\)](#) for additional information.

**void print(unsigned x, uint8\_t y, const char \*buffer, uint8\_t len);**

Print a buffer from (x, y).

**Parameters**

|        |                                                                             |
|--------|-----------------------------------------------------------------------------|
| x      | x coordinate of top-left corner; -1 continues from the last unaligned print |
| y      | y coordinate of top-left corner                                             |
| buffer | characters to print                                                         |
| len    | number of characters to print                                               |

No more than 255 characters can be printed at once. See [print\(unsigned x, uint8\\_t y, char c\)](#) for additional information.

**void print(unsigned x, uint8\_t y, int number, uint8\_t base);**

Print a number from (x, y).

Note that there are no long int overloads.

**Parameters**

|        |                                                                             |
|--------|-----------------------------------------------------------------------------|
| x      | x coordinate of top-left corner; -1 continues from the last unaligned print |
| y      | y coordinate of top-left corner                                             |
| number | number to print                                                             |
| base   | base to print in: $2 \leq \text{base} \leq 36$                              |

**void print(unsigned x, uint8\_t y, unsigned number, uint8\_t base);**

Print a number from (x, y).

Note that there are no long int overloads.

**Parameters**

|        |                                                                             |
|--------|-----------------------------------------------------------------------------|
| x      | x coordinate of top-left corner; -1 continues from the last unaligned print |
| y      | y coordinate of top-left corner                                             |
| number | number to print                                                             |
| base   | base to print in: $2 \leq \text{base} \leq 36$                              |

**void print\_p(const char \*str);**

Print a string from host's ROM.

**Parameters**

|     |                 |
|-----|-----------------|
| str | string to print |
|-----|-----------------|

No more than 255 characters can be printed at once.

**void print\_p(unsigned x, uint8\_t y, const char \*str);**

Print a string from (x, y) from the host's ROM.

**Parameters**

|     |                                                                             |
|-----|-----------------------------------------------------------------------------|
| x   | x coordinate of top-left corner; -1 continues from the last unaligned print |
| y   | y coordinate of top-left corner                                             |
| str | string to print                                                             |

No more than 255 characters can be printed at once.

**void print\_p(unsigned x, uint8\_t y, const char \*buffer, uint8\_t len);**

Print a buffer from (x, y) from the host's ROM.

**Parameters**

|     |                                                                             |
|-----|-----------------------------------------------------------------------------|
| x   | x coordinate of top-left corner; -1 continues from the last unaligned print |
| y   | y coordinate of top-left corner                                             |
| str | characters to print                                                         |
| len | number of characters to print                                               |

**void println(char c);**

Print a character and go to the next line as if [GU3000\\_carriageReturn\(\)](#) and [GU3000\\_lineFeed\(\)](#) were called.

**Parameters**

|   |                    |
|---|--------------------|
| c | character to print |
|---|--------------------|

**void println(const char \*str);**

Print a string and go to the next line as if [GU3000\\_carriageReturn\(\)](#) and [GU3000\\_lineFeed\(\)](#) were called.

**Parameters**

|     |                 |
|-----|-----------------|
| str | string to print |
|-----|-----------------|

**void println(const char \*buffer, size\_t size);**

Print a buffer of the given size and go to the next line as if [GU3000\\_carriageReturn\(\)](#) and [GU3000\\_lineFeed\(\)](#) were called.

**Parameters**

|        |                     |
|--------|---------------------|
| buffer | characters to print |
|--------|---------------------|

|      |                               |
|------|-------------------------------|
| size | number of characters to print |
|------|-------------------------------|

**void println(int number, uint8\_t base);**

Print a number in the given base and go to the next line as if [GU3000\\_carriageReturn\(\)](#) and [GU3000\\_lineFeed\(\)](#) were called.

No leading space or zeros are used.

Each character is printed with [print\(char c\)](#).

**Parameters**

|        |                                                |
|--------|------------------------------------------------|
| number | number to print                                |
| base   | base to print in: $2 \leq \text{base} \leq 36$ |

Arithmetic with a long, unsigned, or constant larger than INT\_MAX will cause the long, unsigned, or unsigned long overload of this function to be used.

**void println(unsigned number, uint8\_t base);**

Print a number in the given base.

No leading space or zeros are used and go to the next line as if [GU3000\\_carriageReturn\(\)](#) and [GU3000\\_lineFeed\(\)](#) were called.

Each character is printed with [print\(char c\)](#).

**Parameters**

|        |                                                |
|--------|------------------------------------------------|
| number | number to print                                |
| base   | base to print in: $2 \leq \text{base} \leq 36$ |

Arithmetic with a long or constant larger than UINT\_MAX will cause the long or unsigned long overload of this function to be used.

**void println(long number, uint8\_t base);**

Print a number in the given base and go to the next line as if [GU3000\\_carriageReturn\(\)](#) and [GU3000\\_lineFeed\(\)](#) were called.

No leading space or zeros are used.

Each character is printed with [print\(char c\)](#).

**Parameters**

|        |                                                |
|--------|------------------------------------------------|
| number | number to print                                |
| base   | base to print in: $2 \leq \text{base} \leq 36$ |

Arithmetic with an unsigned long or constant larger than LONG\_MAX will cause the unsigned long overload of this function to be used.

**void println(unsigned long number, uint8\_t base);**

Print a number in the given base and go to the next line as if [GU3000\\_carriageReturn\(\)](#) and [GU3000\\_lineFeed\(\)](#) were called.

No leading space or zeros are used.

Each character is printed with [print\(char c\)](#).

#### Parameters

|        |                                                |
|--------|------------------------------------------------|
| number | number to print                                |
| base   | base to print in: $2 \leq \text{base} \leq 36$ |

## Character-Based

### **void GU3000\_back();**

Move the cursor back one character position in the current font.

If the cursor is within one character position of the left of the screen, the cursor moves to the last character position of the previous line. If on the first line when this happens, the cursor does not move.

### **void GU3000\_forward();**

Move the cursor forward one character position in the current font.

If the cursor is within one character position of the right of the screen, the cursor moves to the first character position of the next line. If on the last line when this happens, the cursor moves to the home position (0, 0).

### **void GU3000\_lineFeed();**

Move the cursor down one line.

If the cursor is on the last line, then the cursor is moved to the home position (0, 0).

This command is ignored in horizontal scrolling modes.

### **void GU3000\_home();**

Move the cursor to the home position (0, 0).

### **void GU3000\_carriageReturn();**

Move the cursor to the beginning of the current line.

### **void GU3000\_setCursor(unsigned x, unsigned y);**

Set cursor to the given position.

If the position is invalid, the command is ignored.

This command may be used to set the cursor into the hidden memory area.

#### Parameters

|   |                     |
|---|---------------------|
| x | target x coordinate |
| y | target y coordinate |

**void GU3000\_clearLine();**

Clear the current line and set the cursor to the beginning of the line.

**void GU3000\_clearLineEnd();**

Clear the current line from the cursor position to the end of the line. The cursor is not moved.

**void GU3000\_cursorOn();**

Turn the cursor on.

**void GU3000\_cursorOff();**

Turn the cursor off.

## Module Control

**void GU3000\_clearScreen();**

Clear the screen and move the cursor to the home position (0, 0) of the current screen.

If in separate screen mode, only one of the two screens is cleared. If in the hidden memory area, the cursor returns to (0, 0) of the hidden memory area not of the visible screen.

**void GU3000\_init();**

Initialize the module.

**void GU3000\_setScreenBrightness(unsigned level);**

Set screen brightness.

### Parameters

|       |                                                                             |
|-------|-----------------------------------------------------------------------------|
| level | percent of capacity (rounded to next 12.5%): $0 \leq \text{level} \leq 100$ |
|-------|-----------------------------------------------------------------------------|

**void GU3000\_wait(uint8\_t time);**

Stop processing commands for the given time period.

### Parameters

|      |                                 |
|------|---------------------------------|
| time | $\text{time} \times .5\text{s}$ |
|------|---------------------------------|

**void GU3000\_shortWait(uint8\_t time);**

Stop processing commands for the given time period.

**Parameters**

Timing differs per  
module:  $13 \leq T \leq 15$

|      |                        |
|------|------------------------|
| time | $\text{time} \times T$ |
|------|------------------------|

**void GU3000\_displayOn();**

Turn the display on. The screen may fade on.

**void GU3000\_displayOff();**

Turn the display off. The screen may fade off.

Turning the display off is saves more power than simply turning all dots off or setting the brightness to 0% with [GU3000\\_setScreenBrightness\(unsigned level\)](#).

**void GU3000\_displayOff(uint8\_t timer);**

Turn the display off after the given number of minutes. The screen may fade off.

Turning the display off is saves more power than simply turning all dots off or setting the brightness to 0% with [GU3000\\_setScreenBrightness\(unsigned level\)](#).

**Parameters**

|       |                                                                 |
|-------|-----------------------------------------------------------------|
| timer | time to wait before turning the display off ( $\pm 10\%$ error) |
|-------|-----------------------------------------------------------------|

**Fonts**

**void GU3000\_setFontStyle(FontStyle style);**

Set the font style.

**Parameters**

|       |                                           |
|-------|-------------------------------------------|
| style | font style; see <a href="#">FontStyle</a> |
|-------|-------------------------------------------|

**void GU3000\_setFontSize(FontFormat format, uint8\_t x, uint8\_t y);**

Set the style of font.

Characters already on the screen are not affected.

**Parameters**

|        |                                                        |
|--------|--------------------------------------------------------|
| format | size of the characters; see <a href="#">FontFormat</a> |
| x      | width magnification: $1 \leq x \leq 4$                 |
| y      | height magnification: $1 \leq y \leq 4$                |

**void GU3000\_boldOn();**

Turns the bold font style on.

**void GU3000\_boldOff();**

Turns the bold font style off.

**void GU3000\_useCustomChars(bool enable);**

Enables or disables the use of custom characters.

Custom characters are not deleted if this is called with false; they may be used again when custom characters are re-enabled.

**Parameters**

|        |                                                     |
|--------|-----------------------------------------------------|
| enable | true enables custom characters; false disables them |
|--------|-----------------------------------------------------|

**void GU3000\_defineCustomChar(unsigned code, FontFormat format, const uint8\_t \*data);**

Redefine the appearance of the given character code. Redefining a character does not change the appearance of characters already on screen. Custom characters may be redefined multiple times without deleting them.

Only 16 characters may be defined per format.

Custom characters are destroyed when the module is reset or the initialize command is used. Delete characters with [GU3000\\_deleteCustomChar\(unsigned code, FontFormat format\)](#).

Custom characters must be enabled with [GU3000\\_useCustomChars\(bool enable\)](#) to see changes.

**Parameters**

|        |                                                                                                                                                                                                                                                                                        |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| code   | character code<br>$0x20 \leq \text{code} \leq 0xff$ (except $16 \times 16$ and $32 \times 32$ )<br>$0xec40 \leq \text{code} \leq 0xec4f$ for $16 \times 16$ and $32 \times 32$ Japanese<br>$0xfea1 \leq \text{code} \leq 0xfeb0$ for $16 \times 16$ and $32 \times 32$ except Japanese |
| format | size and format of data; see <a href="#">FontFormat</a>                                                                                                                                                                                                                                |
| data   | display data; see the module specification for format                                                                                                                                                                                                                                  |

**void GU3000\_deleteCustomChar(unsigned code, FontFormat format);**

Delete a custom character.

The command is ignored if a character was not defined.

**Parameters**

|      |                                                                                                                                                                                                                                                                                        |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| code | character code<br>$0x20 \leq \text{code} \leq 0xff$ (except $16 \times 16$ and $32 \times 32$ )<br>$0xec40 \leq \text{code} \leq 0xec4f$ for $16 \times 16$ and $32 \times 32$ Japanese<br>$0xfea1 \leq \text{code} \leq 0xfeb0$ for $16 \times 16$ and $32 \times 32$ except Japanese |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|        |                                                         |
|--------|---------------------------------------------------------|
| format | size and format of data; see <a href="#">FontFormat</a> |
|--------|---------------------------------------------------------|

## Encoding

### **void GU3000\_useMultibyteChars(bool enable);**

Enable or disable the use of multibyte character sets.

Use [GU3000\\_setMultibyteCharset\(uint8\\_t code\)](#) to select the character set.

#### **Parameters**

|        |                                                            |
|--------|------------------------------------------------------------|
| enable | true enables multibyte character sets; false disables them |
|--------|------------------------------------------------------------|

Multi-byte characters may only be printed when the 8×16 font is selected. 12×16 may be used for Japanese.

### **void GU3000\_setMultibyteCharset(uint8\_t code);**

Set the multibyte character set.

Use [GU3000\\_useMultibyteChars\(bool enable\)](#) to enable multicharacter sets.

#### **Parameters**

|      |                                                                   |
|------|-------------------------------------------------------------------|
| code | number code for the charset; see <a href="#">MultibyteCharset</a> |
|------|-------------------------------------------------------------------|

Multi-byte characters may only be printed when the 8×16 font is selected. 12×16 may be used for Japanese.

### **void GU3000\_setAsciiVariant(AsciiVariant code);**

Select the national ASCII variant.

The appearance of these characters changes:

|      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|
| 0x23 | 0x24 | 0x40 | 0x5b | 0x5c | 0x5d | 0x5e | 0x60 | 0x7c | 0x7d | 0x7e |
| '#'  | '\$' | '@'  | '['  | '\'  | ']'  | '^'  | '`'  | '{'  | ' '  | '}'  |

Characters already on the screen are not affected.

#### **Parameters**

|      |                                                      |
|------|------------------------------------------------------|
| code | ASCII variant code; see <a href="#">AsciiVariant</a> |
|------|------------------------------------------------------|

### **void GU3000\_setCharset(Charset code);**

Select the character set (code page).

The appearance of characters 0x80 - 0xff changes according to the table for the selected character set.

Characters already on the screen are not affected.

This only affects 8-bit character sets. Use [GU3000\\_setMultibyteCharset\(uint8\\_t code\)](#) to change the multibyte character set.

#### Parameters

|      |                                                 |
|------|-------------------------------------------------|
| code | Character set code; see <a href="#">Charset</a> |
|------|-------------------------------------------------|

## Screen Effects

### **void GU3000\_invertOn();**

Turn inversion on. Dots that would have been lit will be drawn unlit and vice versa.

Text and images already on screen are not affected.

### **void GU3000\_invertOff();**

Turns inversion off.

Text and images already on screen are not affected.

### **void GU3000\_setCompositionMode(CompositionMode mode);**

Determine the way that new images and text are combined with the contents that are already on the screen.

#### Parameters

|      |                                                       |
|------|-------------------------------------------------------|
| mode | composition mode; see <a href="#">CompositionMode</a> |
|------|-------------------------------------------------------|

### **void GU3000\_setScrollMode(ScrollMode mode);**

Select the behavior when text reaches the end of the screen.

No commands are processed while the module is scrolling.

#### Parameters

|      |                                             |
|------|---------------------------------------------|
| mode | scroll mode; see <a href="#">ScrollMode</a> |
|------|---------------------------------------------|

### **void GU3000\_setHorizScrollSpeed(uint8\_t speed);**

Set the delay between scrolling animation steps.

No commands are processed while the module is scrolling. Timing differs per module:  $13 \leq T \leq 15$

#### Parameters

|       |                                                                                                                                |
|-------|--------------------------------------------------------------------------------------------------------------------------------|
| speed | 0 prevents any delay between steps<br>$1 \text{ } T \text{ ms} / 2 \text{ dots}$<br>$(n - 1) \times T \text{ ms} / \text{dot}$ |
|-------|--------------------------------------------------------------------------------------------------------------------------------|

### **void GU3000\_scrollScreen(unsigned x, unsigned y, unsigned count, uint8\_t speed);**

Scroll the contents of the screen by (x, y) leftwards/upwards.

This command only moves the viewing area. The visible screen area is moved left/up and the right / bottom edge now appears in the hidden memory area.

No commands are processed while the module is scrolling.

The cursor remains in place. It does not scroll with the cursor.

Scrolling can be used for page flipping if the module has enough display memory for two full screens. Set the cursor into the hidden memory area and draw the next frame. When rendering is complete, scroll the width of a screen to exchange the hidden memory area with the previous visible memory area. The cursor will remain in the hidden memory area.

#### Parameters

Timing differs per module:  $13 \leq T \leq 15$

|       |                                                           |
|-------|-----------------------------------------------------------|
| x     | number of dots to move left                               |
| y     | number of dots to move up rounded down to the next 8 dots |
| count | number of steps to break animation into                   |
| speed | delay between each step; $\text{speed} \times T$          |

**void GU3000\_blinkScreen(bool enable, bool reverse, uint8\_t on, uint8\_t off, uint8\_t times);**

Enable or disable screen blinking.

#### Parameters

|         |                                                                                                        |
|---------|--------------------------------------------------------------------------------------------------------|
| enable  | true enables screen blinking; false disables it                                                        |
| reverse | true reverses the screen on 'off' cycles; false clears the screen                                      |
| on      | time to display the normal screen image; $\text{on} \times T$                                          |
| off     | time to display the inverted or blank screen image; $\text{on} \times T$                               |
| times   | number of times to blink the screen off; 0 blinks until vfd.GU3000_blinkScreen(false, false, 1, 1, 1); |

Timing differs per module:  $13 \leq T \leq 15$

**void GU3000\_wipeScreen(WipeDirection direction, unsigned speed, uint8\_t filler);**

Slowly fill the screen with the given pattern.

#### Parameters

Timing differs per module:  $13 \leq T \leq 15$

|           |                                                          |
|-----------|----------------------------------------------------------|
| direction | direction of the wipe; see <a href="#">WipeDirection</a> |
| speed     | delay between each step; $\text{speed} \times T$         |
| filler    | bit pattern to fill the screen                           |

**void GU3000\_reverseWipeScreen(WipeDirection direction, unsigned speed, unsigned addr);**

Slowly replace the screen with an image from another part of display memory.

#### Parameters

Timing differs per module:  $13 \leq T \leq 15$

|           |                                                          |
|-----------|----------------------------------------------------------|
| direction | direction of the wipe; see <a href="#">WipeDirection</a> |
| speed     | delay between each step; $\text{speed} \times T$         |
| addr      | address of the image to fill the screen with             |

**void GU3000\_disolveScreen(unsigned speed, unsigned addr);**

Slowly replace the screen with an image from another part of display memory. The image appears in random pieces.

#### Parameters

Timing differs per module:  $13 \leq T \leq 15$

|       |                                                  |
|-------|--------------------------------------------------|
| speed | delay between each step; $\text{speed} \times T$ |
| addr  | address of the image to fill the screen with     |

## Drawing

**void GU3000\_drawImage(unsigned width, uint8\_t height, const uint8\_t \*data);**

Draw image at the cursor from host RAM.

The cursor is not moved.

If height is larger than the screen, then the call will be ignored.

#### Parameters

|        |                                                                                            |
|--------|--------------------------------------------------------------------------------------------|
| width  | width of image; may be less than the width of the image in memory                          |
| height | height of image; must be the height of the image in memory rounded down to the next 8 dots |
| data   | image data; see <a href="#">Image Format</a>                                               |

**void GU3000\_drawImage\_p(unsigned width, uint8\_t height, const uint8\_t \*data);**

Draw image at the cursor from host ROM.

The cursor is not moved.

If height is larger than the screen, then the call will be ignored.

#### Parameters

|        |                                                                                            |
|--------|--------------------------------------------------------------------------------------------|
| width  | width of image; may be less than the width of the image in memory                          |
| height | height of image; must be the height of the image in memory rounded down to the next 8 dots |
| data   | image data; see <a href="#">Image Format</a>                                               |

**void GU3000\_drawImage(ImageMemoryArea area, unsigned long address, uint8\_t srcHeight, unsigned width, uint8\_t height);**

Draw image at the cursor from one of the module's image memory areas.

If height or srcHeight is larger than the

The source image starts from the address given.

screen, then the call will be ignored.

The cursor is not moved.

Parameters

|           |                                                                         |
|-----------|-------------------------------------------------------------------------|
| area      | area in which the bitmap is stored; see <a href="#">ImageMemoryArea</a> |
| address   | address in the memory area                                              |
| srcHeight | height of image in memory memory rounded down to the next 8 dots        |
| width     | width of image; may be less than the width of the image in memory       |
| height    | height of image; may be less than the height of the image in memory     |

**void GU3000\_drawImage(unsigned x, uint8\_t y, ImageMemoryArea area, unsigned long address, uint8\_t srcHeight, unsigned width, uint8\_t height, unsigned offsetx, unsigned offsety);**

Draw image at (x, y) from the module's image memory areas.

If height or srcHeight is larger than the screen, then the call will be ignored.

The source image starts from (offsetx, offsety) with the origin at the byte specified by address.

Parameters

|           |                                                                         |
|-----------|-------------------------------------------------------------------------|
| x         | x coordinate of top-left corner                                         |
| y         | y coordinate of top-left corner                                         |
| area      | area in which the bitmap is stored; see <a href="#">ImageMemoryArea</a> |
| address   | address in the memory area                                              |
| srcHeight | height of image in memory memory rounded down to the next 8 dots        |
| width     | width of image; may be less than the width of the image in memory       |
| height    | height of image; may be less than the height of the image in memory     |
| offsetx   | x coordiante offset from source address                                 |
| offsety   | y coordiante offset from source address                                 |

**void GU3000\_drawImage(unsigned x, uint8\_t y, ImageMemoryArea area, unsigned long address, unsigned width, uint8\_t height);**

Draw image at (x, y) from the module's image memory areas.

If height is larger than the screen, then the call will be ignored. Unlike other functions, this rounds height up since images with heights not divisible by 8 may be drawn.

The source image starts from the byte specified by address.

Parameters

|         |                                                                         |
|---------|-------------------------------------------------------------------------|
| x       | x coordinate of top-left corner                                         |
| y       | y coordinate of top-left corner                                         |
| area    | area in which the bitmap is stored; see <a href="#">ImageMemoryArea</a> |
| address | address in the memory area                                              |
| width   | width of image; may be less than the width of the image in memory       |

|        |                                                      |
|--------|------------------------------------------------------|
| height | height of image rounded <b>up</b> to the next 8 dots |
|--------|------------------------------------------------------|

**void GU3000\_drawImage\_p(unsigned x, uint8\_t y, unsigned width, uint8\_t height, const uint8\_t \*data);**

Draw image at (x, y) from host RAM.

If height is larger than the screen, then the call will be ignored.

**Parameters**

|        |                                                                                            |
|--------|--------------------------------------------------------------------------------------------|
| width  | width of image; may be less than the width of the image in memory                          |
| height | height of image; must be the height of the image in memory rounded down to the next 8 dots |
| data   | image data; see <a href="#">Image Format</a>                                               |

**void GU3000\_drawImage(unsigned x, uint8\_t y, unsigned width, uint8\_t height, const uint8\_t \*data);**

Draw image at (x, y) from host ROM.

If height is larger than the screen, then the call will be ignored.

**Parameters**

|        |                                                                                            |
|--------|--------------------------------------------------------------------------------------------|
| width  | width of image; may be less than the width of the image in memory                          |
| height | height of image; must be the height of the image in memory rounded down to the next 8 dots |
| data   | image data; see <a href="#">Image Format</a>                                               |

**void GU3000\_scrollImage(ImageMemoryArea area, unsigned long address, uint8\_t srcHeight, unsigned width, uint8\_t height, uint8\_t speed);**

Draw image at the cursor from one of the module's image memory areas.

The source image starts from the address given.

The cursor is not moved.

If height or srcHeight is larger than the screen, then the call will be ignored.

**Parameters**

|           |                                                                         |
|-----------|-------------------------------------------------------------------------|
| area      | area in which the bitmap is stored; see <a href="#">ImageMemoryArea</a> |
| address   | address in the memory area                                              |
| srcHeight | height of image in memory memory rounded down to the next 8 dots        |
| width     | width of image; may be less than the width of the image in memory       |
| height    | height of image; may be less than the height of the image in memory     |

**void GU3000\_dot(unsigned x, unsigned y, bool on=true);**

Set the pixel at (x, y).

**Parameters**

|    |                                                |
|----|------------------------------------------------|
| x  | x coordinate                                   |
| y  | y coordinate                                   |
| on | true turns the dot on; false turns the dot off |

**void GU3000\_shape(Shape shape, unsigned x0, unsigned y0, unsigned x1, unsigned y1, bool on=true);**

Draws a shape from (x0, y0) to (x1, y1).

#### Parameters

|       |                                                |
|-------|------------------------------------------------|
| shape | shape to draw; see <a href="#">Shape</a>       |
| x0    | x coordinate of the top-left corner            |
| y0    | y coordinate of the top-left corner            |
| x1    | x coordinate of the bottom-right corner        |
| y1    | y coordinate of the bottom-right corner        |
| on    | true turns the dot on; false turns the dot off |

**void GU3000\_defineImage(ImageMemoryArea area, unsigned addr, unsigned width, uint8\_t height, const uint8\_t \*data);**

Define an image from host's RAM in one of the module's image memory areas.

See the module's spec for the limits on image memory area's limits. If height is larger than the screen, then the call will be ignored.

#### Parameters

|        |                                                                           |
|--------|---------------------------------------------------------------------------|
| area   | memory area to define the image in; see <a href="#">ImageMemoryArea</a> . |
| addr   | address in the image memory area                                          |
| width  | width of the image to define                                              |
| height | height of the image to define rounded down to the next 8 dots             |

**void GU3000\_defineImage\_p(ImageMemoryArea area, unsigned addr, unsigned width, uint8\_t height, const uint8\_t \*data);**

Define an image from host's ROM in one of the module's image memory areas.

See the module's spec for the limits on image memory area's limits. If height is larger than the screen, then the call will be ignored.

#### Parameters

|        |                                                                           |
|--------|---------------------------------------------------------------------------|
| area   | memory area to define the image in; see <a href="#">ImageMemoryArea</a> . |
| addr   | address in the image memory area                                          |
| width  | width of the image to define                                              |
| height | height of the image to define rounded down to the next 8 dots             |
| data   | image data; see <a href="#">Image Format</a>                              |

## Window and Screen

**void GU3000\_selectWindow(uint8\_t window);**

Select the base window or one of the user-defined windows.

**Parameters**

|        |                                                                                  |
|--------|----------------------------------------------------------------------------------|
| window | 0 selects the base window $1 \leq \text{window} \leq \text{user-defined window}$ |
|--------|----------------------------------------------------------------------------------|

**void GU3000\_defineWindow(uint8\_t window, unsigned x, unsigned y, unsigned width, unsigned height);**

Define or redefine a window from (x,y) to (x+width, y+height).

**Parameters**

|        |                                                                      |
|--------|----------------------------------------------------------------------|
| window | $1 \leq \text{window} \leq \text{user-defined window}$               |
| x      | x coordinate of upper-left corner                                    |
| y      | y coordinate of upper-left corner                                    |
| width  | width of the window; may not be past the edge of the visible screen  |
| height | height of the window; may not be past the edge of the visible screen |

**void GU3000\_deleteWindow(uint8\_t window);**

Delete a user-defined window.

The command is ignored if the user-defined window is not defined.

**Parameters**

|        |                                                        |
|--------|--------------------------------------------------------|
| window | $1 \leq \text{window} \leq \text{user-defined window}$ |
|--------|--------------------------------------------------------|

**void GU3000\_joinScreens();**

Treat the display memory as one screen.

Text scrolls and wraps when the cursor reaches the right end of the hidden memory area.

**void GU3000\_separateScreens();**

Treat the display memory as two separate screens: visible screen area and hidden memory area.

## Appendix A: Command Support

---

|                   |                                     |
|-------------------|-------------------------------------|
| Character display | <code>void print(char c);</code>    |
| Back Space        | <code>void GU3000_back();</code>    |
| Horizontal Tab    | <code>void GU3000_forward();</code> |

|                                     |                                                                                                   |
|-------------------------------------|---------------------------------------------------------------------------------------------------|
| Line feed                           | <code>void GU3000_lineFeed();</code>                                                              |
| Home                                | <code>void GU3000_home();</code>                                                                  |
| Carriage Return                     | <code>void GU3000_carriageReturn();</code>                                                        |
| Display clear                       | <code>void GU3000_clearScreen();</code>                                                           |
| Line Clear                          | <code>void GU3000_clearLine();</code>                                                             |
| Line end Clear                      | <code>void GU3000_clearLineEnd();</code>                                                          |
| Cursor set                          | <code>void GU3000_setCursor(unsigned x, unsigned y);</code>                                       |
| Cursor display                      | <code>void GU3000_cursorOn();</code>                                                              |
| Initialize display                  | <code>void GU3000_init();</code>                                                                  |
| Font size select                    | <code>void GU3000_setFontSize(FontFormat format, uint8_t x, uint8_t y);</code>                    |
| 2-byte character                    | <code>void GU3000_useMultibyteChars(bool enable);</code>                                          |
| 2-byte character type               | <code>void GU3000_setMultibyteCharset(uint8_t code);</code>                                       |
| Download character ON/OFF           | <code>void GU3000_useCustomChars(bool enable);</code>                                             |
| Download character definition       | <code>void GU3000_defineCustomChar(unsigned code, FontFormat format, const uint8_t *data);</code> |
| Download character delete           | <code>void GU3000_deleteCustomChar(unsigned code, FontFormat format);</code>                      |
| 16×16 Download character definition | <code>void GU3000_defineCustomChar(unsigned code, FontFormat format, const uint8_t *data);</code> |
| 16×16 Download character delete     | <code>void GU3000_deleteCustomChar(unsigned code, FontFormat format);</code>                      |
| 32×32 Download character definition | <code>void GU3000_defineCustomChar(unsigned code, FontFormat format, const uint8_t *data);</code> |
| 32×32 Download character delete     | <code>void GU3000_deleteCustomChar(unsigned code, FontFormat format);</code>                      |
| Download character save             |                                                                                                   |
| Download character restore          |                                                                                                   |
| International font set              | <code>void GU3000_setAsciiVariant(AsciiVariant code);</code>                                      |
| Character table type                | <code>void GU3000_setCharset(Charset code);</code>                                                |
| Font Width                          | <code>void GU3000_setFontStyle(FontStyle style);</code>                                           |
| Font Magnification                  | <code>void GU3000_setFontSize(FontFormat format, uint8_t x, uint8_t y);</code>                    |
| Bold character                      | <code>void GU3000_boldOn();</code>                                                                |
| FROM user font definition           |                                                                                                   |
| FROM extension font definition      |                                                                                                   |
| FROM extended font                  |                                                                                                   |
| Over-write mode                     | <code>void GU3000_setScrollMode(ScrollMode mode);</code>                                          |
| Vertical scroll mode                | <code>void GU3000_setScrollMode(ScrollMode mode);</code>                                          |
| Horizontal scroll mode              | <code>void GU3000_setScrollMode(ScrollMode mode);</code>                                          |

|                                       |                                                                                                                                                                                                |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Horizontal scroll mode scroll ON      | <code>void GU3000_setScrollMode(ScrollMode mode);</code>                                                                                                                                       |
| Horizontal scroll speed               | <code>void GU3000_setHorizScrollSpeed(uint8_t speed);</code>                                                                                                                                   |
| Reverse display                       | <code>void GU3000_invertOn();</code>                                                                                                                                                           |
| Write mixture display mode            | <code>void GU3000_setCompositionMode(CompositionMode mode);</code>                                                                                                                             |
| Brightness level setting              | <code>void GU3000_setScreenBrightness(unsigned level);</code>                                                                                                                                  |
| Wait                                  | <code>void GU3000_wait(uint8_t time);</code>                                                                                                                                                   |
| Short Wait                            | <code>void GU3000_shortWait(uint8_t time);</code>                                                                                                                                              |
| Scroll display action                 | <code>void GU3000_scrollScreen(unsigned x, unsigned y, unsigned count, uint8_t speed);</code>                                                                                                  |
| Blink                                 | <code>void GU3000_blinkScreen(bool enable, bool reverse, uint8_t on, uint8_t off, uint8_t times);</code>                                                                                       |
| Curtain display action                | <code>void GU3000_wipeScreen(WipeDirection direction, unsigned speed, uint8_t filler);</code>                                                                                                  |
| Spring display action                 | <code>void GU3000_reverseWipeScreen(WipeDirection direction, unsigned speed, unsigned addr);</code>                                                                                            |
| Random display action                 | <code>void GU3000_disolveScreen(unsigned speed, unsigned addr);</code>                                                                                                                         |
| Screen saver                          | <code>void GU3000_displayOn();</code>                                                                                                                                                          |
| Display power auto-OFF time           | <code>void GU3000_displayOff(uint8_t timer);</code>                                                                                                                                            |
| Real-time bit image display           | <code>void GU3000_drawImage(unsigned width, uint8_t height, const uint8_t *data);</code>                                                                                                       |
| Downloaded bit image display          | <code>void GU3000_drawImage(ImageMemoryArea area, unsigned long address, uint8_t srcHeight, unsigned width, uint8_t height);</code>                                                            |
| Dot unit downloaded bit image display | <code>void GU3000_drawImage(unsigned x, uint8_t y, ImageMemoryArea area, unsigned long address, uint8_t srcHeight, unsigned width, uint8_t height, unsigned offsetx, unsigned offsety);</code> |
| Dot unit real-time bit image display  | <code>void GU3000_drawImage(unsigned x, uint8_t y, unsigned width, uint8_t height, const uint8_t *data);</code>                                                                                |
| Dot unit character display            | <code>void print(unsigned x, uint8_t y, const char *str);</code>                                                                                                                               |
| RAM bit image definition              | <code>void GU3000_defineImage(ImageMemoryArea area, unsigned addr, unsigned width, uint8_t height, const uint8_t *data);</code>                                                                |
| FROM bit image definition             |                                                                                                                                                                                                |
| Downloaded bit image scroll display   | <code>void GU3000_scrollImage(ImageMemoryArea area, unsigned long address, uint8_t srcHeight, unsigned width, uint8_t height, uint8_t speed);</code>                                           |
| Window select                         | <code>void GU3000_selectWindow(uint8_t window);</code>                                                                                                                                         |
| User Window define / cancel           | <code>void GU3000_defineWindow(uint8_t window, unsigned x, unsigned y, unsigned width, unsigned height);</code>                                                                                |
| Write screen mode select              | <code>void GU3000_joinScreens();</code>                                                                                                                                                        |
| Memory re-write mode                  |                                                                                                                                                                                                |

|                                 |                                                                                                                |
|---------------------------------|----------------------------------------------------------------------------------------------------------------|
| Dot drawing                     | <code>void GU3000_dot(unsigned x, unsigned y, bool on=true);</code>                                            |
| Line/Box pattern drawing        | <code>void GU3000_shape(Shape shape, unsigned x0, unsigned y0, unsigned x1, unsigned y1, bool on=true);</code> |
| User setup mode start           |                                                                                                                |
| User setup mode end             |                                                                                                                |
| I/O Port Input / Output setting |                                                                                                                |
| I/O Port Output                 |                                                                                                                |
| I/O Port Input                  |                                                                                                                |
| RAM Macro define / delete       |                                                                                                                |
| FROM Macro define / delete      |                                                                                                                |
| Macro end condition             |                                                                                                                |
| Macro execution                 |                                                                                                                |
| General-purpose memory store    |                                                                                                                |
| General-purpose memory transfer |                                                                                                                |
| Memory SW setting               |                                                                                                                |
| Memory SW data send             |                                                                                                                |
| Display status send             |                                                                                                                |
| RS-232 serial settings          |                                                                                                                |